

InRideMart

AI-Powered In-Ride Shopping Concierge

A full-stack commerce experience that helps passengers discover useful, cab-available convenience products during a ride, with an AI assistant that turns travel context into explainable suggestions.

Built as an existing Java Spring Boot microservice monorepo with a Next.js frontend. The project reuses service ownership across authentication, catalog, cart, checkout, payment, and AI orchestration.

Executive Summary

InRideMart addresses the small but time-sensitive needs that surface in transit: a dead phone, a long airport journey, hunger, rain, or a forgotten essential. It keeps discovery and purchase close to the passenger while preserving the Catalog service as the product source of truth.

1

Discover
Browse the active catalog and assigned demo cab inventory.

2

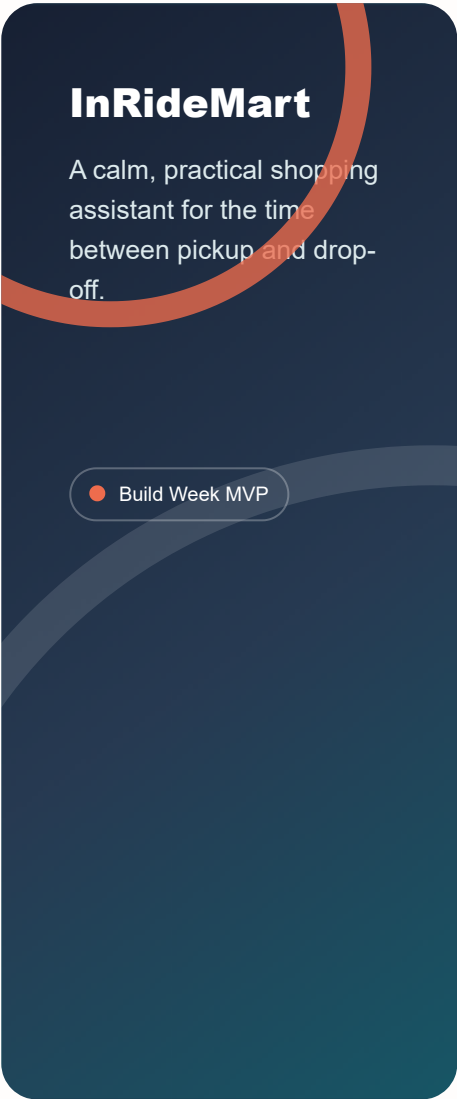
Decide
Ask the assistant for relevant products and optional bundles.

3

Complete
Use cart, checkout, mock payment, and order tracking.

Contents

Executive summary	1	Architecture and operations	3
Problem, solution, features	2	AI capabilities and roadmap	4



OVERVIEW

Problem, Solution, and Features

Problem Statement

Passengers often realize they need a charger, drink, snack, travel comfort item, or rain essential only after a ride has started. Traditional quick-commerce services are not optimized for the short time window, current journey context, or inventory that is immediately accessible in the vehicle.

Solution Overview

InRideMart brings a catalog-led shopping flow into the ride. A conversational AI layer understands intent and context, then recommends only products returned by the existing catalog and available in the assigned demo cab inventory.

Implemented MVP Capabilities

AI

Conversational Shopping

Natural requests, clarifying questions, budget-aware suggestions, and catalog-grounded explanations.

CB

Travel Bundles

Contextual bundle suggestions such as airport, charging, rainy-day, and road-trip essentials.

CI

Cab Inventory View

Frontend filters the existing catalog to the assigned demo cab inventory. It is not a live driver inventory integration.

CT

Cart and Checkout

Authenticated cart ownership, product validation, checkout, and order creation through dedicated services.

PM

Mock Payment

UPI, credit card, debit card, and wallet choices with a production-like successful-payment experience.

TR

Order Tracking

Order progression from placed to confirmed, handed over, and completed.

Scope note: All product availability, prices, and names come from the Catalog service. AI does not invent products, prices, stock, or payment outcomes.

How the Platform Runs

Next.js provides the mobile-first web experience and a same-origin API proxy. The backend uses independently owned Spring Boot services; PostgreSQL, Redis, and Kafka are started through Docker Compose for local development.

Next.js Frontend :3000



API Proxy



Spring Boot Services



PostgreSQL / Redis / Kafka

Service Ports

SERVICE	PORT
Auth	8081
Customer	8082
Catalog	8083
Cart	8084
Order	8085
AI	8086
Payment	8087

API Surface

/api/v1/auth	register, login, current user
/api/v1/customers	customer profiles
/api/v1/catalog/products	browse and search
/api/v1/carts/me	cart, items, checkout
/api/v1/orders	orders and tracking
/api/v1/payments	mock payment and status
/api/v1/ai	chat and translation

Local Setup

```
git clone <REPLACE-WITH-REPOSITORY-URL>
cd InRideMart
$env:OPENAI_API_KEY = "<your-key>" # optional; fallback remains available
.\start-inridemart.ps1
```

The start script sets local development variables, starts Docker Compose, opens service processes, and runs the frontend. PostgreSQL is exposed on host port 5433; Redis uses 6379; Kafka uses 9092.

AI Capabilities and Next Steps

How GPT-5.6 Is Used

The dedicated AI service uses the OpenAI Responses API for conversational reasoning, intent detection, follow-up questions, and multilingual translation. The model is configured through environment variables; no secret is embedded in source.

Known shopping intents are matched against catalog data, with context including budget, destination, weather, journey duration, purpose, and available cab inventory. GPT is never asked to create product facts outside that catalog.

Resilient Behavior

When an OpenAI key is unavailable or a live call fails, the AI service uses deterministic, catalog-grounded handling and makes that fallback explicit. Empty or ambiguous requests prompt for clarification rather than producing a generic product list.

The multilingual assistant uses GPT when configured and local phrase support only when live translation is unavailable.

Future Roadmap

INVENTORY	Live driver and cab inventory synchronization, replenishment signals, and availability updates.
COMMERCE	Real payment-gateway integration, refunds, receipt delivery, loyalty rewards, and richer fulfillment states.
MOBILITY	Ride booking integration, destination signals, driver workflows, and a native mobile client using the same backend APIs.
AI	Voice interaction, stronger preference memory with consent, and predictive recommendations based on opt-in trip history.

OpenAPI Documentation

SERVICE	SWAGGER UI
Auth	http://localhost:8081/swagger-ui.html
Customer	http://localhost:8082/swagger-ui.html
Catalog	http://localhost:8083/swagger-ui.html
Order	http://localhost:8085/swagger-ui.html
AI	http://localhost:8086/swagger-ui.html

Cart and Payment APIs are available through their service endpoints but do not currently expose a Springdoc Swagger UI in the repository configuration.